AutoCloud-Agent

Multi-Agent Reinforcement Learning for Cloud Resource Management

K Chaitanya (25CS60R28)

Rahul K (25CS60R16)

Sri Lalitha Rama Siddesh Devarakonda (25CS60R80)

April 2026

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

Talk Outline

1. Problem & Motivation
2. System Architecture
3. Three RL Agents + I-PPO
4. Workload Forecaster
5. Safety Coordinator
6. Cloud Simulator
7. Results & Ablation
8. Conclusion

The Problem: Two Competing Objectives

Elasticity

Scale up *before* demand spikes
SLA: P95 latency $< 500\,\text{ms}$
VM boot delay: 30–90 s

vs.

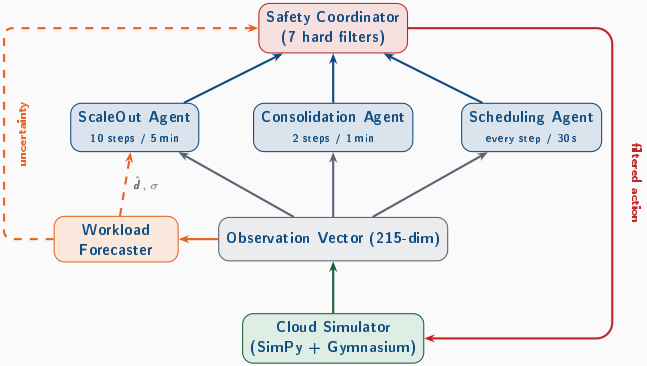
Cost Minimization

Drain idle VMs aggressively
Pack jobs tightly
Never pay for idle cores

Why existing methods fail:
Kubernetes HPA / AWS Auto Scaling are *reactive* — they respond *after* CPU crosses a threshold. By then, VMs need 60s to boot and SLA is already violated.

Why single-agent RL fails:
Joint action space $= 3 \times 2^{20} \times 5 \approx 15$ M combinations. Too large to learn efficiently from scratch.

System Architecture



Three Specialized Agents — Temporal Hierarchy

ScaleOut Agent

Action: $\{0, +1, +2\}$ VMs

Space: **3**

Every **10 steps** (5 min)

Capacity planning

Consolidation Agent

Action: drain mask $\{0, 1\}^{20}$

Space: 2^{20}

Every **2 steps** (1 min)

Cost optimization

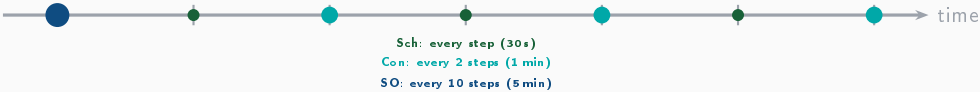
Scheduling Agent

Action: priority $\{0..4\}$

Space: **5**

Every **step** (30 s)

Job dispatch



Key: joint space $3 \times 2^{20} \times 5 \approx 15\text{M}$ $\rightarrow$ three independent spaces of size **3**, 2^{20}, **5**

Independent PPO (I-PPO) — How Each Agent Learns

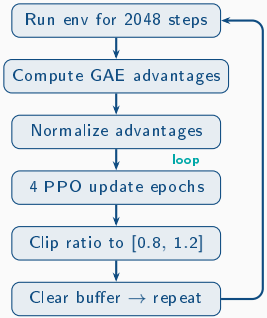
PPO Clipped Objective

$$L^{\text{CLIP}} = \mathbb{E}\Big[\min\Big(r_t\hat{A}_t,\ \text{clip}(r_t,\ 1-\epsilon,\ 1+\epsilon)\hat{A}_t\Big)\Big]$$

where $r_t = \dfrac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

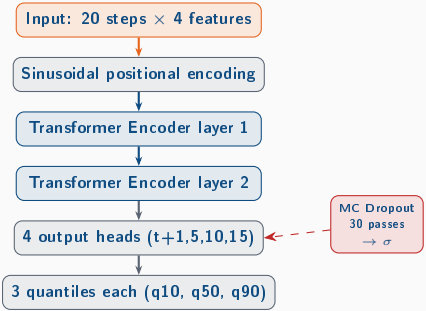
MLP Architecture (each agent):
215 $\rightarrow$ 512 $\xrightarrow{\text{LN+ReLU}}$ 256 $\xrightarrow{\text{LN+ReLU}}$ 128 $\rightarrow$ head

Separate **Actor** (policy) + **Critic** (value)



Hyperparams: $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, lr $= 3 \times 10^{-4}$, entropy $0.01 \rightarrow 0.001$

Workload Forecaster — Transformer + MC Dropout



Why MC Dropout?

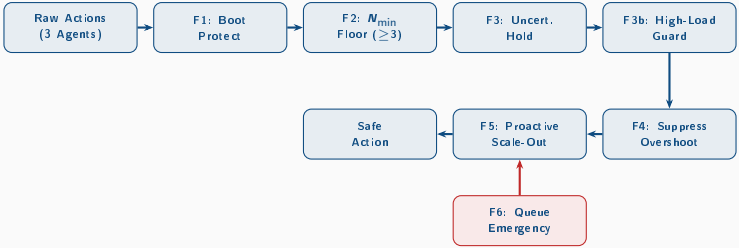
Keep dropout **on** at inference.
Run 30 forward passes → 30 different predictions.

$\sigma = \mathrm{var}(30 \text{ outputs})$

Low σ → model confident → act normally
High σ → model uncertain → Safety Coordinator blocks drains and forces scale-up

Forecaster output injected into obs[200:215]:
means $[\hat{d}_{t+1}, \hat{d}_{t+5}, \hat{d}_{t+10}, \hat{d}_{t+15}]$
sigmas $[\sigma_{t+1}, \sigma_{t+5}, \sigma_{t+10}, \sigma_{t+15}]$

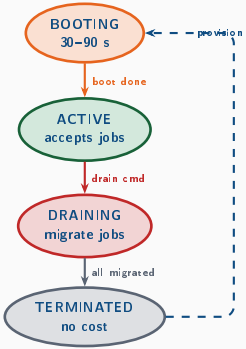
Safety Coordinator — 7-Filter Pipeline



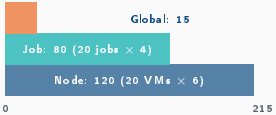
Key insight: Hard constraints in the *reward* only penalise violations — they do not *prevent* them. The coordinator runs *after* the agents and *guarantees* no unsafe action ever reaches the simulator, regardless of how well (or poorly) the agents are trained.

Cloud Simulator — VM Lifecycle & Observation Space

VM State Machine



Observation Vector (215 dims)



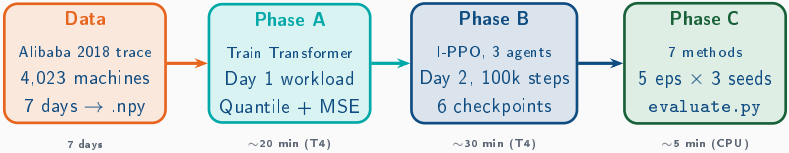
Node (6/VM): CPU, mem, age, booting, active, draining

Job (4/job): service, wait, priority, deadline urgency

Global (15): active VMs, queue len, CPU/mem, $\hat{d}$, σ

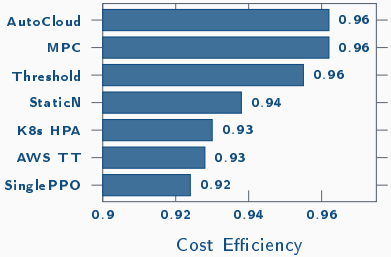
SimPy discrete-event + Gymnasium API. 30 s/step, 120 steps = 1 h.

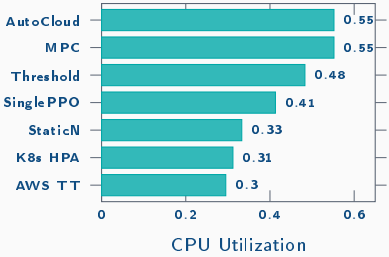
Training Pipeline



Why a simulator? Training RL needs 100k+ environment steps. Resetting real cloud infrastructure 500 times is impossible — bad actions during training would cause real outages. SimPy lets us train safely at scale.

Results — Alibaba Cluster Trace 2018

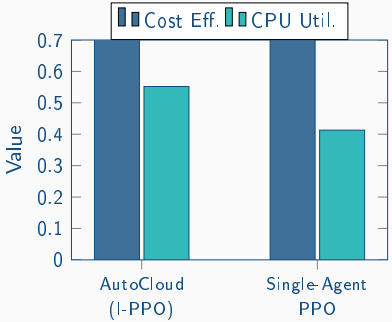




All 7 methods achieve 100% SLA compliance.
Differentiator is cost efficiency and CPU utilization.

AutoCloud beats K8s HPA by +3.4% cost, +77% CPU util.
Matches MPC while being fully *learned*, not hand-tuned.

Ablation — Why Multi-Agent Decomposition?



Why I-PPO wins

1. **Smaller action spaces**
   Each agent: 3, 2^{20}, or 5 — not 15M joint
2. **Temporal hierarchy**
   Fast decisions (30s) can't mix with slow (5min) ones
3. **Per-agent rewards**
   Clearer learning signal per subtask
4. **Credit assignment**
   Easier to debug — which agent caused a violation?

+4.1% cost efficiency
+33.7% CPU utilization
over single-agent RL

Stress Testing — 4 Out-of-Distribution Scenarios

(1) Ramp-Up
$1\times \rightarrow 3\times$ linear ramp
Tests provisioning

(2) Early Shock
$4\times$ spike at step 15
Filters 5+6 fire

(3) Choppy Plateau
$2.5\times \pm$ noise
No VM thrashing

(4) Trough + Recovery
$0.3\times$ then $2.5\times$
Avoid over-drain

Why stress test? These patterns are *out-of-distribution* — the agents were trained on the Alibaba Day 2 trace. Stress tests verify the Safety Coordinator and learned policy generalise to unseen workload shapes.

Live Demo — `python demo.py`

`standard`
RL agent vs
one chosen baseline
step-by-step

`shootout`
RL vs all
5 baselines
ranked table

`stress`
Extreme workload
RL vs chosen baseline
4 scenarios

`ablation`
3-agent I-PPO
vs Single-Agent
PPO

Each step displays:

- forecaster predictions $\pm\sigma$
- peak queue depth
- VM state dots (booting/active/draining)
- agent decisions + Safety Coordinator overrides
- live cost & cumulative SLA rate

Quick start:

```
python demo.py   (wizard)
python demo.py -mode stress
python demo.py -mode shootout
python demo.py -mode ablation
python demo.py -no-pause -speed fast
```

Conclusion

Three Contributions

1. Multi-Agent Decomposition (I-PPO)
3 agents at different temporal frequencies — 15M joint space → three manageable spaces

2. Uncertainty-Aware Forecasting
Transformer + MC Dropout → proactive scaling before the spike arrives

3. Safety Coordinator
7 hard-constraint filters guarantee no unsafe action ever reaches the simulator

96.2% cost efficiency
55.2% CPU utilization
100% SLA compliance

Matches **MPC** (best classical)
Beats **K8s HPA** by 3.4% cost
+4.1% cost, **+33.7%** CPU over single-agent RL

Refs: Schulman et al. (PPO, 2017)
Vaswani et al. (Transformers, 2017)
Gal & Ghahramani (MC Dropout, 2016)
Zhang et al. (AWARE, ATC 2023)